Database Table Reference

How Trading Strategy Lab uses its tables. Traced from the SQLAlchemy models, Alembic revisions, stores, and API routes. Column types are in CURRENT_DATABASE_ARCHITECTURE.md.

The research database maps **10** tables plus the `trade_journal` view. The live database maps **19** tables. The market database maps **5** tables. All of them have a current writer. Nothing in the head schema is an unused leftover table. Live tables that `0002_live_schema` created inside the research database are dropped by `0004_split_live`.

Analysis date: 2026-10-08.

Executive summary

Table	Database	Purpose	Feature
`strategies`	Research	Identity of a strategy. Updated for the current version and for archive	Lab
`ingest_runs`	Research	One extract, including model and raw output	Ingest
`strategy_versions`	Research	Immutable Markdown and rules	Lab
`reports`	Research	Review Crew verdict and payload	Review
`simulation_runs`	Research	One backtest or forward window	Review
`sim_trades`	Research	One simulated trade	Journal
`trade_reviews`	Research	Review of a simulated trade	Journal
`strategy_improvements`	Research	Proposed rule change and its evidence	Improvements
`improvement_decisions`	Research	Human decision on a proposal	Improvements
`audit_events`	Research	Hash-chained research audit log	Audit
`deployed_strategies`	Live	Frozen copy of an approved version	Approval
`broker_accounts`	Live	Broker, masked account ref, mode	Brokers
`deployments`	Live	Frozen strategy plus risk limits on an account	Deploy
`deployment_events`	Live	Status history. The latest row is the state	Deploy
`trade_intents`	Live	A decision to enter, exit, protect, or block	Trade Crew
`orders`	Live	One order sent to the adapter	Orders
`order_events`	Live	Status changes while an order is polled	Orders
`fills`	Live	A fill	Orders
`live_trades`	Live	A closed or partially closed live trade	Journal
`broker_charges`	Live	Charges the broker reported	Orders
`position_snapshots`	Live	Marked open positions after a tick	Equity
`capital_events`	Live	Deposit or withdrawal	Money
`drawdown_events`	Live	A step on the drawdown ladder	Drawdown
`live_trade_reviews`	Live	Review of a live trade, including LLM lessons	Journal

Table	Database	Purpose	Feature
`live_audit_events`	Live	Hash-chained live audit log	Audit
`trade_opinions`	Live	LLM take, reduce, or skip	Opinions
`phantom_trades`	Live	What removed lots would have done	Opinions
`broker_credentials`	Live	Latest encrypted value of one credential field	Credentials
`crew_heartbeats`	Live	Proof the crew ticked	Supervisor
`index_bars`	Market	Index OHLC, VIX, IV rank	Backtest and live context
`option_eod`	Market	Option closes	Backtest
`import_files`	Market	Files already imported	Importers
`market_events`	Market	Event calendar	Blackouts and time exits
`news_items`	Market	Headlines	LLM context

---

1. Database overview

Application code opens three SQLAlchemy engines (`backend/app/db/session.py` and `backend/app/market/store.py`).

- **Research** holds ingest, versions, reports, simulations, reviews, improvements, and `audit_events`.
- **Live** holds everything that happens after approval. It does not foreign-key back to research. Approval copies the version text, rules, and hashes.
- **Market** holds NSE reference data. It is not append-only. Reports store a fingerprint of what was imported.

`GET /api/journal` reads the research view `trade_journal` and live `live_trades`, then returns one list. Live rows are skipped when the live database is not migrated.

Alembic bookkeeping (`alembic_version`) is omitted below.

---

2. Table documentation

strategies

`strategies` is the only research row that changes after insert. `current_version` moves forward when a version is saved. `POST /api/strategies/{id}/archive` sets `archived_at`. Deletes are rejected.

- **Written by:** `StrategyStore` in `backend/app/strategies/store.py`.
- **Read by:** strategy list and detail routes, review, approval preview.

ingest_runs

One row per extract attempt. `extractor`, `model`, and `raw_output` are the reproducibility record for the LLM step.

- **Written by:** `IngestRunStore` from `POST /api/ingest/extract` and the demo path.
- **Read by:** `GET /api/ingest-runs/{id}`.

strategy_versions

The Markdown that was approved or reviewed. Later edits are new rows. `content_sha256` covers the text. Rules live in `frontmatter`.

- **Written by:** ingest, and by an improvement decision of `applied`.
- **Read by:** review, approval preview, and the UI version picker.

reports

One Review Crew execution. `inputs` is the provenance blob (costs, settings, market-data fingerprint, engine version, rules version, git commit, app version). `payload` holds the verdict narrative and `hard_stop_sweep`. `verdict_label` is the scenario verdict only.

- **Written by:** `ReportStore` after `run_review_crew`.
- **Read by:** report routes, approval (a completed report must exist).

simulation_runs

One scenario in one mode (`backtest` or `forward`). Unique on `(report_id, scenario_index, mode)`.

- **Written by:** the report store from the crew's `sim_runs`.
- **Read by:** `GET /api/reports/{id}/simulations`.

sim_trades

The trade journal's research side: why, how, vision, and outcome. `outcome_label` is `win`, `loss`, or `scratch`.

- **Written by:** the report store.
- **Read by:** `GET /api/reports/{id}/trades`, the CSV route, and the `trade_journal` view.

trade_reviews

Reviews of simulated trades only (`ck sim_trade_only`). The crew inserts `reviewer_type = agent`. Humans insert through `POST /api/trades/sim/{trade_id}/reviews`. A correction is a new row with `supersedes_review_id`.

- **Written by:** `backend/app/agents/review_crew/reviewer.py` and `JournalStore`.
- **Read by:** trade review list and the journal's latest verdict and tags.

strategy_improvements

A proposed change. `evidence` includes the trade keys. The hard-stop recommendation is one of these rows. An LLM rationale is appended to `rationale` when a key is configured. The proposal itself is deterministic.

- **Written by:** `backend/app/agents/review_crew/improver.py` during the review run.
- **Read by:** `GET /api/strategies/{id}/improvements`.

improvement_decisions

`accepted`, `rejected`, or `applied`. `applied` requires Markdown and sets `resulting_version_id`. `rejected` and `applied` are final in the API.

- **Written by:** `POST /api/improvements/{id}/decisions`.
- **Read by:** the improvements list, which derives status from these rows.

audit_events

Every research write that the stores choose to audit. `hash` chains to `prev_hash`. `GET /api/audit/verify` recomputes the chain.

- **Written by:** `backend/app/audit/log.py`.
- **Read by:** `GET /api/audit` and `/verify`.

deployed_strategies

The copy the Trade Crew is allowed to trust. Includes Markdown, `rules`, `content_sha256`, and `rules_sha256`.

- **Written by:** `backend/app/live/approvals.py` in the same transaction as the deployment.
- **Read by:** the crew's verify step and deployment detail.

broker_accounts

A named account. `broker` must be registered and must support `mode`. Creating a live account requires a token.

- **Written by:** `POST /api/live/broker-accounts`.
- **Read by:** approval and the deploy page.

deployments

The approved risk limits, including drawdown and policy, frozen as JSON. Status is not a column. It is the latest `deployment_events.event`.

- **Written by:** approval.
- **Read by:** crew, supervisor, `/api/live/deployments`.

deployment_events

Append-only status. Trading requires the latest event to be `started`. `stopped` and `halted` are final. Floor and veto notices are also events and do not by themselves change that rule; the crew or supervisor writes a real status event when it pauses or starts.

- **Written by:** approval, start, pause, stop, the crew on halt, and the supervisor on floor resume.
- **Read by:** every gate check.

trade_intents

The decision record. `approval` is `auto` (orders sent), `blocked` (a risk check failed), or `vetoed` (the LLM skip). Protective stops use a `protect` intent.

- **Written by:** `LiveStore` from the Trade Crew.
- **Read by:** `GET /api/live/deployments/{id}/intents`, reconciliation, exits.

orders, order_events, fills

The order lifecycle in `backend/app/trading/orders.py`. Hedges are ordered before shorts. Status polls become `order_events`. Fills are unique per `(order_id, broker_trade_id)`.

- **Written by:** the order manager inside the crew.
- **Read by:** `GET /api/live/orders/{id}`, charge and position logic.

live_trades

A completed close, or a partial close when `partial` is true. `exit_reason` records `kill_switch`, `broker_stop`, `unwind`, and the strategy or policy exit that fired.

- **Written by:** the crew when a position closes.
- **Read by:** journal, equity, opinion evaluation, `GET /api/live/live-trades`.

broker_charges

Charges the adapter reports, separate from the estimated cost model on the fill.

- **Written by:** the crew when the adapter returns charges.
- **Read by:** live trade actual cost.

position_snapshots

Open positions and their mark, written after a tick. Equity uses the latest snapshot's unrealized P&L.

- **Written by:** the crew.
- **Read by:** `GET /api/live/equity`, `GET /api/live/positions`, drawdown.

capital_events

Deposits and withdrawals after approval. Amounts are positive. Kind says which way. Equity and peak move by the same amount.

- **Written by:** `POST /api/live/deployments/{id}/capital-events`.
- **Read by:** equity and `GET .../capital-events`.

drawdown_events

One row each time the ladder level changes, including a return to normal.

- **Written by:** `backend/app/live/drawdown.py` from the crew tick.
- **Read by:** `GET /api/live/deployments/{id}/drawdown`.

live_trade_reviews

Same review shape as `trade_reviews`, for live trades. LLM lessons use `reviewer_type = agent` and `reviewer = llm:<model>`.

- **Written by:** `POST /api/trades/live/{trade_id}/reviews` and the crew lesson writer.
- **Read by:** review history and later opinions (`lesson_review_ids`).

live_audit_events

The live database's own hash chain, including credential fingerprints and approval details. Values of secrets are not stored.

- **Written by:** `LiveStore`.
- **Read by:** `GET /api/live/audit` and `/verify`.

trade_opinions

Stored for `shadow` and `veto`. `applied` is true only when the opinion changed the lots that were sent. Errors are stored and the rules stand.

- **Written by:** `backend/app/agents/trade_crew/opinion.py`.

- **Read by:** `GET /api/live/opinions` and the evaluation route.

phantom_trades

The lots a veto skipped or reduced, marked until the exit rule would have closed them. Written once, at exit.

- **Written by:** the crew when opinions are in veto.
- **Read by:** `GET /api/live/phantom-trades` and opinion evaluation.

broker_credentials

Append-only secret store. The latest row for a field is the value. A revoked row drops that field so lookup continues to `BROKER_<BROKER>_<FIELD>` in the environment.

- **Written by:** `PUT` and `DELETE /api/live/broker-accounts/{id}/credentials`.
- **Read by:** adapter construction. The GET route returns field, source, fingerprint, and time. It does not return ciphertext.

crew_heartbeats

One row per tick with feed, outcome, and pid.

- **Written by:** the crew at the end of a tick.
- **Read by:** deployment list (`crew_running`) and the supervisor's missed-heartbeat alert.

index_bars, option_eod, import_files

NSE end-of-day import. `import_files` is how a re-run skips a file. Index highs and lows feed the liquidity-sweep entry fields. VIX closes feed IV rank and the odds gate.

- **Written by:** `backend/scripts/import_nse_eod.py`.
- **Read by:** the simulator, the policy backtest, and the broker feed's previous-day range.

market_events

RBI, budget, FOMC, results, and optional board meetings.

- **Written by:** `backend/scripts/import_events.py`.
- **Read by:** entry blackouts and policy event exits. An empty calendar fails closed when a blackout rule is set.

news_items

Headlines only, with publish time and fetch time. Feeds are not a backfill.

- **Written by:** `backend/scripts/fetch_news.py`.
- **Read by:** LLM opinion context, filtered to items published at or before the decision.

---

3. Group tables by domain

Domain	Tables
Strategy identity	`strategies`, `strategy_versions`, `ingest_runs`
Review	`reports`, `simulation_runs`, `sim_trades`, `trade_reviews`, `strategy_improvements`, `improvement_decisions`

Domain	Tables
Research audit	audit_events
Approval	deployed_strategies, broker_accounts, deployments, deployment_events
Execution	trade_intents, orders, order_events, fills, live_trades, broker_charges
Risk and money	position_snapshots, capital_events, drawdown_events
LLM opinions	trade_opinions, phantom_trades, live_trade_reviews
Live operations	broker_credentials, crew_heartbeats, live_audit_events
Market reference	index_bars, option_eod, import_files, market_events, news_items

---

4. Application flow to table mapping

Flow	Tables written
Ingest	ingest_runs, strategies, strategy_versions, audit_events
Review	reports, simulation_runs, sim_trades, trade_reviews, strategy_improvements, audit_events
Apply improvement	improvement_decisions, strategy_versions, strategies (version pointer), audit_events
Approve	deployed_strategies, deployments, deployment_events, live_audit_events, and one research audit_events row
Trade Crew tick	trade_intents, orders, order_events, fills, live_trades, position_snapshots, drawdown_events, crew_heartbeats, live_audit_events. Opinions and phantom trades only when that feature is on
Kill switch	deployment_events, then order cancels and, if flatten, live_trades
Credential rotate	broker_credentials, live_audit_events
NSE import	index_bars, option_eod, import_files

---

5. Table dependency map

Research parents, in insert order: strategies and ingest_runs, then strategy_versions, then reports, then simulation_runs, then sim_trades, then trade_reviews. Improvements point at a strategy, a version, and optionally a report. Decisions point at an improvement and optionally a new version.

Live parents, in insert order: broker_accounts and deployed_strategies, then deployments, then events, intents, orders, fills, and trades. Credentials and heartbeats point at accounts and deployments. They can exist before any trade.

Market tables are independent of both.

---

6. Important identifiers

Identifier	Meaning
`strategies.id`	Stable strategy id. Copied onto live rows. Not a foreign key there
`strategy_versions.id`	Integer snapshot id. Approval stores a copy
`content_sha256`	Hash of the Markdown. Crew halts if the frozen copy no longer matches
`rules_sha256`	Hash of the rules object. The UI must send it back as `confirm_rules_sha256`
`deployments.id`	The id the supervisor and the crew process use
`client_order_id`	Idempotent order id unique in `orders`
`audit hash`	`sha256(prev_hash + canonical JSON)`. One chain per audited database
`import_files.sha256`	File already imported into the market database
Credential fingerprint	Shown in API and audit. Not the secret

---

7. Vector data

There is no embedding column, no `tsvector`, and no vector extension. Search is not a feature of this schema. Strategy text is stored as Markdown and JSON rules.

---

8. Tables that are no longer in the research database

`0002_live_schema` created broker, deployment, order, and live-trade tables in the research database. `0004_split_live` drops them and rebuilds `trade_journal` over simulated trades only. A database migrated to head does not contain those research-side live tables. The live database's `0001_live` is the replacement, and later live revisions add opinions, drawdown, partial trades, credentials, and heartbeats.

---

9. Source references

Area	File
Models	`backend/app/db/models/`
Journal merge	`backend/app/journal/store.py`
Live writes	`backend/app/live/store.py`
Approval	`backend/app/live/approvals.py`
Market create	`backend/app/market/store.py`
View SQL	`backend/alembic/versions/0004_split_live.py`

Documentation verification

- Table names match `__tablename__` on the ORM classes exported from `backend/app/db/models/__init__.py` and `backend/app/market/models.py`.
- `LIVE_APPEND_ONLY_TABLES` lists the same 19 live tables.
- Research append-only list matches the nine append-only research tables. `strategies` is no-delete rather than append-only.
- No service writes a table that is absent from this list.